Prova de Aptidão Profissional (PAP)

Curso Profissional de Técnico/a de Gestão de Equipamentos Informáticos

Segurança Máxima

Trabalho realizado por: Rafael Correia nº20
Turma: 12º PTEI
Ano Letivo: 2025/2026
Data: 6 de março de 2026

Agradecimentos

Agradeço a todos os professores, em particular ao Professor Ricardo Pires e aos colegas pelo apoio durante o desenvolvimento do projeto, assim como à escola pela oportunidade de aplicar conhecimentos teóricos em situações práticas.

Índice

1. Introdução
2. Objetivo
3. Justificação
4. Desenvolvimento do Projeto
5. Algoritmo
6. Materiais utilizados
7. Resultados
8. Conclusão

1. Introdução

A segurança constitui uma preocupação constante na sociedade atual, quer em ambientes residenciais, comerciais ou institucionais. O avanço tecnológico tem permitido o desenvolvimento de soluções cada vez mais acessíveis, inteligentes e eficazes para a proteção de bens e pessoas.

No âmbito desta Prova de Aptidão Profissional, foi desenvolvido um sistema de videovigilância com deteção de movimento, recorrendo a uma câmara e a uma aplicação implementada na liguagem de programação Python. O sistema permite identificar movimentos num espaço monitorizado e acionar automaticamente um alarme sonoro, bem como iniciar a gravação de vídeo.

O objetivo principal é reforçar a segurança de um determinado espaço, permitindo a deteção precoce de intrusões e contribuindo para a prevenção de incidentes.

2. Objetivo

Os principais objetivos deste projeto são:

- Desenvolver um sistema de segurança funcional e automatizado;
- Implementar um mecanismo de deteção de movimento através de processamento de imagem;
- Acionar automaticamente um alarme sonoro quando for detetado movimento;
- Gravar automaticamente vídeo sempre que seja identificada atividade suspeita;
- Criar uma solução acessível, de baixo custo e adaptável a diferentes contextos.

3. Justificação

O aumento de situações de furto e invasão em residências, estabelecimentos comerciais e instituições reforça a necessidade de implementar medidas de segurança preventivas.

A motivação para a realização deste projeto surgiu da necessidade de aplicar, em contexto real, os conhecimentos adquiridos ao longo do Curso Profissional de Técnico/a de Gestão de Equipamentos Informáticos, nomeadamente nas áreas de programação, hardware e sistemas informáticos.

Além disso, este sistema apresenta a vantagem de ser adaptável a diferentes contextos (habitações, lojas, escritórios ou armazéns), constituindo uma solução prática, económica e personalizável.

4. Desenvolvimento do Projeto

Tarefa	Início	Fim
Levantamento dos Requisitos	1/10/2025	3/10/2025
Planeamento do Propjeto	2/10/2025	6/10/2025
Implementação de um Protótipo	6/10/2025	07/11/2025
Desenvolvimento do Projeto	8/11/2025	24/11/2025
Testes	25/11/2025	12/12/0205
Relatório	12/11/2025	03/03/2026

Fig. 01 – Diagrama de Gantt.

O desenvolvimento do projeto foi dividido em várias etapas técnicas, descritas de seguida.

4.1 Inicialização da Câmara

O programa inicia tentando estabelecer ligação com a câmara de vídeo. Caso a ligação não seja possível, é apresentada uma mensagem de erro e a aplicação é encerrada, garantindo assim um funcionamento controlado e seguro.

```python
import cv2
import os
import winsound
import datetime

cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Erro ao aceder à câmera.")
    exit()
else:
    print("Câmera acedida com sucesso.")
```

4.2 Criação da Pasta de Armazenamento

Após a verificação da câmara, o sistema cria automaticamente uma pasta destinada ao armazenamento dos vídeos capturados. Esta organização permite manter os ficheiros estruturados e facilmente acessíveis.

```python
# Pasta para salvar os vídeos
os.makedirs("videos", exist_ok=True)
```

4.3 Captura da Primeira Frame

Após a criação da pasta, o código tenta capturar uma frame, se não for possível é apresentada uma mensagem de erro e, a ligação com a câmara é encerrada e o programa termina.

```python
ret, frame1 = cap.read()
if not ret:
    print("Erro ao capturar a frame.")
    cap.release()
    exit()
```

4.4 Captura da Segunda Frame

Após a captura da primeira frame é capturada a segunda frame, para que mais tarde seja possível detetar movimento.

```python
ret, frame2 = cap.read()
if not ret:
    print("Erro ao capturar a frame.")
    cap.release()
    exit()
```

4.5 Configuração de Parâmetros de Vídeo

Nesta fase, são definidos os seguintes parâmetros:

- Formato do vídeo (codec XVID);
- Número de frames por segundo (FPS);
- Dimensão das frames;
- Variáveis de controlo da gravação;
- Tempo máximo permitido sem deteção de movimento.

```
# Configurações do vídeo de saída
fourcc = cv2.VideoWriter_fourcc(*'XVID')
fps = 20.0
frame_size = (int(cap.get(3)), int(cap.get(4)))

recording = False
video_writer = None
still_frames = 0
still_frames = 20  # ~1 segundo sem movimento antes de parar de gravar
```

4.6 Processamento de Imagem e Deteção de Movimento

O sistema compara continuamente duas frames consecutivas, realizando os seguintes procedimentos:

- Cálculo da diferença entre imagens;
- Conversão da imagem para escala de cinzentos;
- Aplicação de desfocagem *(blur)* para reduzir ruído;
- Binarização da imagem;
- Aplicação de erosão e dilatação para eliminar ruído residual;
- Deteção de contornos;
- Análise do tamanho dos contornos para validação de movimento.

Quando é identificado um contorno com dimensão significativa:

- O movimento é assinalado na imagem;
- É emitido um alarme sonoro;
- A gravação de vídeo é iniciada;
- É criado automaticamente um ficheiro com data e hora no nome.

Sempre que há movimento, o contador de tempo parado é reiniciado. Caso não seja detetado movimento durante um determinado período, a gravação é interrompida automaticamente.

```python
while cap.isOpened():
    diff = cv2.absdiff(frame1, frame2)
    cv2.imshow("Imagem de diferenca", diff)  # Exibe a imagem de diferença para debug

    gray = cv2.cvtColor(diff, cv2.COLOR_BGR2GRAY)
    cv2.imshow("Imagem em escala de cinza", gray)  # Exibe a imagem em escala de cinza para debug

    blur = cv2.GaussianBlur(gray, (5, 5), 0)
    cv2.imshow("Imagem Gaussiana", blur)  # Exibe a imagem borrada para debug

    _, thresh = cv2.threshold(blur, 20, 255, cv2.THRESH_BINARY)
    cv2.imshow("Imagem a preto e branco", thresh)  # Exibe a imagem de threshold para debug

    eroded = cv2.erode(thresh, None, iterations=3)
    cv2.imshow("Imagem com erosao", eroded)  # Exibe a imagem de threshold para debug

    dilated = cv2.dilate(eroded, None, iterations=5)
    cv2.imshow("Imagem com dilatacao", dilated)  # Exibe a imagem de threshold para debug

    contours, _ = cv2.findContours(dilated, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

    movimento_detectado = False
```

Durante a gravação, cada frame é guardada no ficheiro de vídeo. O sistema apresenta ainda a imagem em tempo real no ecrã.

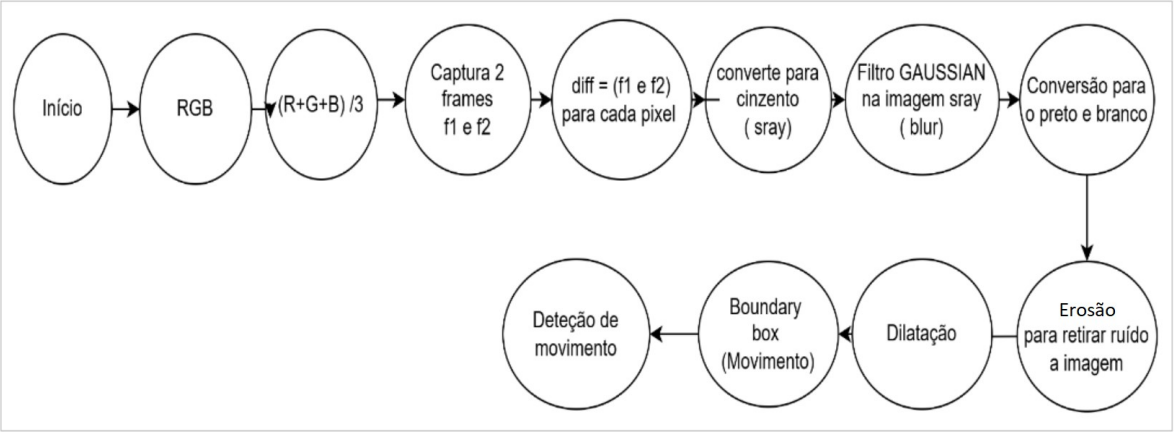
```python
for contour in contours:
    if cv2.contourArea(contour) > 2000:
        movimento_detectado = True
        x, y, w, h = cv2.boundingRect(contour)
        cv2.rectangle(frame1, (x, y), (x + w, y + h), (0, 255, 0), 2)

# Controle de gravação
if movimento_detectado:
    print("Movimento detectado!")
    winsound.Beep(1000, 100)

    if not recording:
        # Inicia nova gravação
        nome_arquivo = datetime.datetime.now().strftime("videos/%Y-%m-%d_%H-%M-%S.avi")
        video_writer = cv2.VideoWriter(nome_arquivo, fourcc, fps, frame_size)
        recording = True
        print(f"🔴 recording: {nome_arquivo}")
```

```
        still_frames = 0
    else:
        if recording:
            still_frames += 1
            if still_frames > still_frames:
                # Para a gravação
                recording = False
                video_writer.release()
                video_writer = None
                print("🟢 Gravação finalizada.")
```

O programa termina caso ocorra falha na câmara ou se o utilizador pressionar a tecla “q”. No final, são libertados os recursos da câmara e todas as janelas são encerradas corretamente.

```
90
91      # Salva frame no vídeo se estiver recording
92      if recording and video_writer:
93          video_writer.write(frame1)
94
95      cv2.imshow("Camera", frame1)
96
97      frame1 = frame2
98      ret, frame2 = cap.read()
99      if not ret:
100         break
101
102     if cv2.waitKey(1) == ord('q'):
103         break
104
105 # Libera tudo ao sair
106 if video_writer:
107     video_writer.release()
108
109 cap.release()
110 cv2.destroyAllWindows()
```

5. Algoritmo



6. Materiais Utilizados

- Computador com sistema operativo Windows;
- Câmara de vídeo (webcam);
- Linguagem de programação Python;
- Bibliotecas de processamento de imagem (OpenCV);
- Colunas de som para emissão do alarme.

6. Resultados

O sistema apresentou um funcionamento satisfatório durante os testes realizados. Sempre que é detetado movimento, o alarme é acionado imediatamente e a gravação é iniciada automaticamente.

O alerta sonoro revela-se eficaz para chamar a atenção de pessoas próximas, funcionando como elemento dissuasor de comportamentos indevidos.

O projeto demonstrou ser uma solução de baixo custo, funcional e eficiente, com potencial para futuras melhorias, como:

- Armazenamento em na núvem *(cloud);*

- Envio de notificações para dispositivos móveis;
- Integração com aplicação móvel;
- Implementação de reconhecimento facial.

7. Conclusão

A realização desta Prova de Aptidão Profissional permitiu consolidar e aplicar conhecimentos técnicos adquiridos ao longo do curso, nomeadamente nas áreas de programação, redes e sistemas informáticos.

O sistema desenvolvido constitui uma solução prática e acessível para reforço da segurança em diversos contextos, como residências e estabelecimentos comerciais.

Apesar de funcional, o projeto poderá ser melhorado futuramente com a integração de novas funcionalidades, tornando-o mais completo, robusto e adaptado às exigências atuais de segurança digital.